

Snatcher: Apple Find My Network Exposes Your Lost Devices to Strangers

Zhenyu Ren*

zrenax@connect.ust.hk

The Hong Kong University of Science and Technology
Hong Kong, China

Boya Liu

bliuby@connect.ust.hk

The Hong Kong University of Science and Technology
Hong Kong, China

Yanbo Zhang*

yanbozhang@ust.hk

The Hong Kong University of Science and Technology
Hong Kong, China

Mo Li[†]

lim@ust.hk

The Hong Kong University of Science and Technology
Hong Kong, China

A Artifact Appendix

A.1 Abstract

Snatcher demonstrates that the mechanisms making Apple’s Find My network effective for owners also expose lost Apple devices to unauthorized discovery, tracking, and physical theft by strangers. The attack relies solely on a commodity Android smartphone and requires no specialized hardware. The paper formalizes a progressive three-level attack: (*Level 1*) Acoustic direction finding; (*Level 2*) Passive RSSI-IMU navigation; and (*Level 3*) De-anonymization via spatial-temporal clustering.

The released artifact includes the SnatchAPP Android application for discovering and actuating nearby lost devices, two ESP32 firmware images as evaluation targets, and representative raw data traces with offline analysis code to verify the navigation attacks safely. The artifact is packaged as a Git repository (also archived on Zenodo), is licensed under AGPL-3.0, and targets the *Artifacts Available*, *Artifacts Evaluated (Functional and Reusable)*, and *Results Reproduced* badges.

A.2 Description & Requirements

The repository is laid out as follows:

Snatcher/	
SnatchAPP/	# sound-play app (paper Table 1)
esp32_airtag_mac_rotation/	# mimicked AirTag (paper Fig 11)
esp32_sound_maker/	# sound trigger fw (paper Table 1)
reproduce/	# example traces + analysis (Python)
1_timeout_disconnect_distance/	# lost-state (paper Fig 3)
2_mac_rotation/	# MAC rotation (paper Table 2)
3_acoustic_direction/	# Level 1 direction (paper Fig 6)
4_rssi_navigation/	# Level 2 RSSI-IMU nav (paper Fig 8)
5_clustering/	# Level 3 clustering (paper Fig 11)
6_max_distance/	# attack range (paper Fig 14,15)
7_navigation_traces/	# end-to-end traces (paper Fig 17)
doc/	# this artifact appendix
README.md LICENSE	# quick start + AGPL-3.0

The top-level README gives a quick start. SnatchAPP and esp32_sound_maker actuate the non-owner sound trigger (paper Table 1); esp32_airtag_mac_rotation is the mimicked-AirTag target for Level-3 clustering. The reproduce/ folder holds seven self-contained examples and is the primary, hardware-free verification path: each pairs a small data/ trace with Python

scripts that print a result and write a figure, reproducing one named result of the paper (named in the tree above and in each experiment below). In total they reproduce the paper’s **Figures 3, 6, 8, 11, 14, 15, and 17** and **Tables 1 and 2** — the lost-state threat model, the three attack levels, the maximum attack range, and the end-to-end navigation cost.

A.2.1 How to access.

- **Archived (permanent):** Zenodo concept DOI (resolves to the latest version), <https://doi.org/10.5281/zenodo.20693210>.
- **Development version:** <https://github.com/rzy0901/Snatcher>.
- **Anonymized mirror:** <https://anonymous.4open.science/r/Snatcher-4D25>.

A.2.2 Hardware dependencies.

- **Trace-based reproduction (E1–E7):** none — any commodity PC with Python suffices.
- **Attacker device (E0):** one Android smartphone running Android 7.0 (API 24) or newer, with Bluetooth Low Energy, a stereo microphone (the UNPROCESSED audio source must be supported for Level 1), and a 9-axis IMU.
- **Evaluation targets (E0):** two ESP32-family boards (we used the ESP32-WROVER-Kit; ESP32-S3/C3 also work) — one for the mimicked-AirTag firmware, one for the sound trigger. Optionally an AirTag/AirPods (Level 1) and an iPhone/Apple Watch (Level 2), *owned by the evaluator, signed into an Apple ID and placed in the Separated (Lost) state*, to validate against production hardware.
- USB cables to flash the ESP32 boards and to retrieve logs from the phone. No GPU is required.

A.2.3 Software dependencies.

- **Python 3.8+** with numpy, scipy, pandas, and matplotlib (reproduce/requirements.txt).
- *Optional, to build the live components:* Android Studio¹ (Hedgehog 2023.1.1+, compileSdk 34, JDK 17, adb) and ESP-IDF² v4.4+.

¹<https://developer.android.com/studio>

²<https://github.com/espressif/esp-idf>

*Zhenyu Ren and Yanbo Zhang contributed equally to this work.

[†]Corresponding author.

- The Android build pulls MPAndroidChart and AndroidX via Gradle; the ESP32 firmware derives from OpenHaystack (mimicked AirTag) and AirGuard (sound trigger).

A.2.4 Benchmarks. All traces are self-collected with SnatchAPP, which logs synchronized CSV streams—BLE scans (MAC, RSSI, payload, timestamp), IMU samples (accelerometer/gyroscope/magnetometer, fused orientation, PDR position, step count), and navigation RSSI (RSSI, phone heading, reconstructed position, target heading)—plus raw stereo PCM audio with per-recording metadata. Each example ships, in its data/ folder, the processed trace it consumes, so the analysis runs without collecting new measurements. The bulkier raw scan logs behind these traces are retained locally (available on request), while 1_timeout_disconnect_distance/, 2_mac_rotation/, and 6_max_distance/ also ship a representative raw capture under raw/ so the raw-to-trace derivation can be inspected. No personal or third-party user data are included.

A.2.5 Security, privacy, and ethical concerns. Snatcher is offensive — it can discover, actuate, and physically localize Apple Find My devices that are not the operator’s. Evaluate it *only* against devices you own (commercial Apple devices you control, or the provided ESP32 lab targets); never trigger sound on, connect to, or track a third party’s device. To avoid misuse, the released SnatchAPP omits the on-device navigation- and clustering-assistance modules; the trace-based reproduction still validates those algorithms offline from the shipped logs and performs no live attack. Running it is otherwise safe: no destructive steps, no privileged access, no security mechanisms disabled.

A.3 Set Up

A *prebuilt* SnatchAPP APK (part (A)) lets reviewers run the live on-device attack (E0) in about a minute with no build — *we encourage trying it*. Building from source (parts (A)/(B)) is optional, and the trace-based reproduction (E1–E7) runs from the provided traces and Python code (part (C)) on any PC.

A.3.1 Installation. (A) Android application. The *fastest path is the prebuilt, signed APK — no build needed*: download SnatchAPP.apk³ and install it on any Android 7.0+ phone (open it on the device and allow “install unknown apps”, or adb install -r SnatchAPP.apk). To build from source instead, open SnatchAPP/ in Android Studio (*Run* installs it), or:

```
cd SnatchAPP
./gradlew assembleRelease
adb install -r app/build/outputs/apk/release/app-release.apk
```

On first launch, grant the requested permissions (Bluetooth scan/connect, fine location, microphone, physical activity, and storage). Figure 1 shows the partially-released app running from Android Studio.

(B) ESP32 firmware (optional). For each firmware, source the ESP-IDF environment and, via idf.py menuconfig, set the Bluetooth controller to *BLE Only* and enable Bluedroid. Then build, flash, and monitor:

```
cd esp32_airtag_mac_rotation # or esp32_sound_maker
idf.py set-target esp32
```

³<https://github.com/rzy0901/Snatcher/raw/main/SnatchAPP/SnatchAPP.apk>

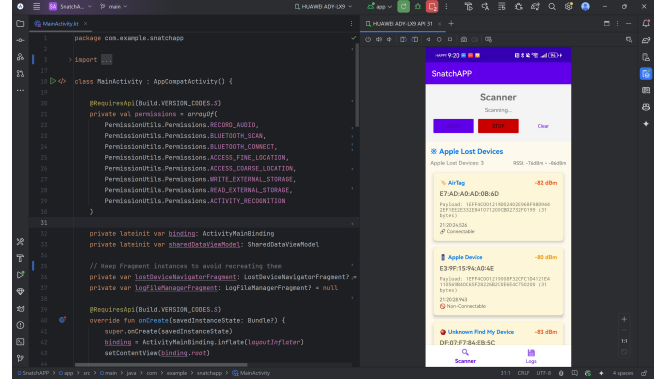


Figure 1: The partially-released SnatchAPP built and run from Android Studio. The Scanner screen lists nearby Apple lost devices with their type, MAC, RSSI, and Find My payload. Only lost-device discovery and the non-owner sound trigger are included; the navigation- and clustering-assistance modules are withheld to mitigate misuse.

```
idf.py menuconfig # set BLE Only + Bluedroid
idf.py -p <PORT> flash monitor
```

In esp32_airtag_mac_rotation/main/openhaystack_main.c, set DEVICE_ID to distinguish co-located mimicked devices and configure the MAC address rotation period (paper range: 10–60 s). In esp32_sound_maker/main/main.c, set device_type and target_mac to the target accessory (its MAC can be read with SnatchAPP or nRF Connect⁴).

(C) Reproduction environment.

```
cd reproduce
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

A.3.2 Basic test. After installing the reproduction environment, run the acoustic example as a quick functionality check:

```
cd reproduce/3_acoustic_direction && python acoustic_example.py
```

It band-passes the recording, cross-correlates it with the chirp template to detect the AirTag sound arrivals, and prints their level (≈ -42.8 dB), confirming the toolchain is correctly installed. With the prebuilt APK installed, E0 (below) is the live on-device demo.

A.4 Evaluation Workflow

The evaluation comprises one functional check (E0) and seven reproduction experiments (E1–E7). E1–E7 run from the provided data and need no Apple hardware; the recommended E0 demo needs only an Android phone with the prebuilt APK — its target is the lost Apple devices already broadcasting nearby, or one you control (an ESP32 board, or an Apple device in the Separated/Lost state). The remaining aggregate field results (the per-level navigation success rates, paper Figure 16, and the averaged end-to-end navigation times) require extensive real-world trials and are out of automated scope; the artifact provides the implementation and logging to repeat them.

⁴<https://www.nordicsemi.com/Products/Development-tools/nRF-Connect-for-mobile>

Results: a two-panel figure (Figure 4). (a) the lost-state timeout per device on a log axis — AirTag ~12 min, AirPods longer but variable, Apple Watch ~13 s; (b) the owner-to-device disconnect distance stays at ~10–20 m across four indoor scenes. These reproduce Figure 3 of the paper.

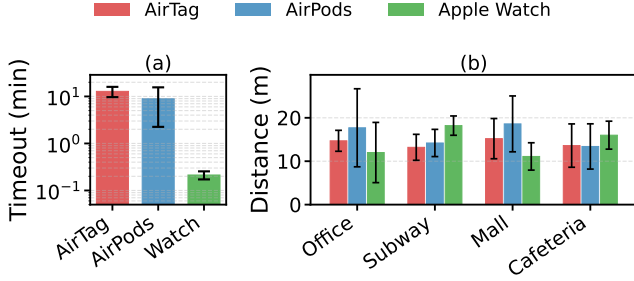


Figure 4: E1, lost-state onset (paper Figure 3): per-device time to enter the lost state (a, log axis) and the owner-to-device disconnect distance across four indoor scenes (b).

(E2) MAC-address rotation period (paper Table 2) [5 person-min]. The Find My address-rotation period per device — Vulnerability III, which bounds how long one address can be followed. **Preparation:** `cd reproduce/2_mac_rotation`.

Execution: `python make_table.py`.

Results: a per-device table (Table 1) that reproduces paper Table 2: Apple accessories hold one MAC for ~24 h, while iPhone and Apple Watch rotate every ~15–36 min. `make_table.py` also writes `mac_rotation.pdf` (a log-scale boxplot of the per-address spread), and `derive_durations.py` re-derives the iPhone column from the shipped capture. The long static window on accessories is what makes Level-3 tracking practical.

Table 1: E2 output, reproducing paper Table 2: measured Find My MAC-address rotation period per device.

Category	Device	MAC rotation period
Apple Accessories	AirPods Pro	~24 h (daily, ~4 am)
	AirTag	~24 h (daily, ~4 am)
Apple Devices	iPhone	fixed ~15 min
	Apple Watch	19–36 min

(E3) Acoustic direction finding (Level 1, paper Figure 6) [10 person-min + <5 compute-min]. The input is the phone’s top-mic channel (data/example_top.pcm, 16-bit mono PCM at 44.1 kHz) and the model chirp template (data/template.wav). **Preparation:** `cd reproduce/3_acoustic_direction`.

Execution:

```
python plot_waveform_spectrogram.py # waveform + spectrogram
python acoustic_example.py          # arrival detection + dB
python plot_direction_polar.py      # direction polar
```

Results: `plot_waveform_spectrogram.py` shows the chirp energy around 2.6–2.8 kHz (Figure 5a). `acoustic_example.py` band-passes both signals to that band, cross-correlates to detect the arrivals, and takes the RMS over each in dB (≈ -42.8 dB; Figure 5b). The four body orientations give four dB values, which

`plot_direction_polar.py` turns into the contrast-enhanced score $S_k = \alpha A_k + \beta (A_k - A_{opp(k)})$ ($\alpha=0.3, \beta=0.7$); the heading $\arg \max_k S_k$ falls at the *Front* by body shadowing (Figure 5c), reproducing the paper’s Level 1 result.

(E4) RSSI-IMU navigation (Level 2, paper Figure 8) [10 person-min]. SnatchAPP logs, per BLE reading, the target’s RSSI with the phone heading and the IMU-derived Pedestrian-Dead-Reckoning (PDR) position and step count (column format in the folder’s README); two sample logs are provided. **Preparation:** `cd reproduce/4_rssi_navigation`.

Execution:

```
python plot_trace.py # data/nav_trace_1.csv
python plot_trace.py --log data/nav_trace_2.csv
```

Results: the script picks the target device (the frequently-seen MAC with the highest peak RSSI), reconstructs the walked path from the logged PDR position, and colours it by RSSI. The path converges on the target as RSSI rises (e.g. $-81 \rightarrow -39$ dBm; Figure 6), reconstructing the paper’s Level 2 navigation trace.

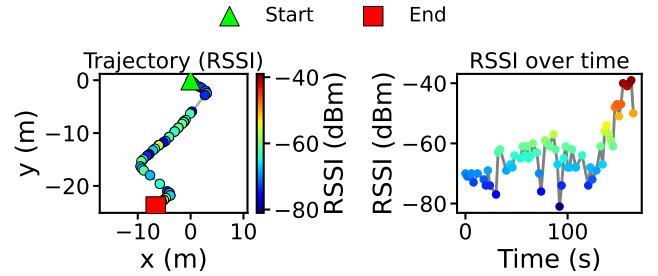


Figure 6: E4, navigation trace: the walked PDR trajectory coloured by RSSI (left) and RSSI over time (right), converging on the target as RSSI rises.

(E5) Spatial-temporal clustering (Level 3, paper Figure 11) [10 person-min]. Co-located mimicked AirTags (the provided ESP32 firmware) at different distances. For exact ground truth the script applies the MAC-address rotation in software, each device rotating independently (the firmware does the same on-device); the clustering runs blind, on timing and RSSI only. **Preparation:** `cd reproduce/5_clustering`.

Execution:

```
python cluster.py --log data/clustering_log_3dev.csv # 3 devices
python cluster.py # 2 devices
```

Results: the script runs the paper’s two stages. *Static profiling* colours the temporal-conflict graph — a device cannot advertise two MACs at once, so overlapping MACs seed different clusters — and reads off each device’s ~30 s rotation period. *Dynamic stitching* then attaches each later MAC to the one cluster whose previous MAC has already ended, breaking ties by RSSI and period. On the three-device log (Figure 7), whose RSSI ranges overlap and cross, the temporal constraint still recovers all three trajectories, defeating MAC rotation as in the paper’s Level 3 result.

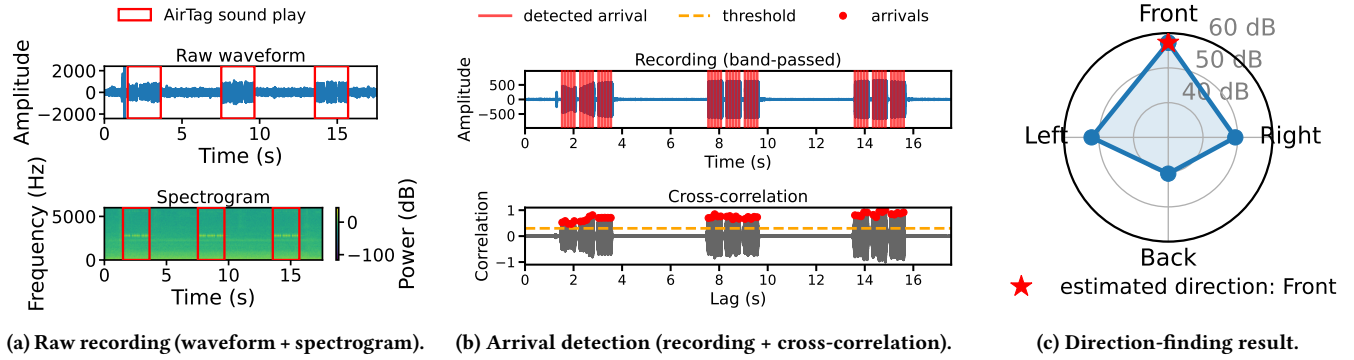


Figure 5: E3, acoustic direction finding (Level 1): the top-mic recording (a), band-passed and cross-correlated with the chirp template, gives the arrival level (≈ -42.8 dB) (b); four body orientations yield the direction estimate, here *Front* (c).

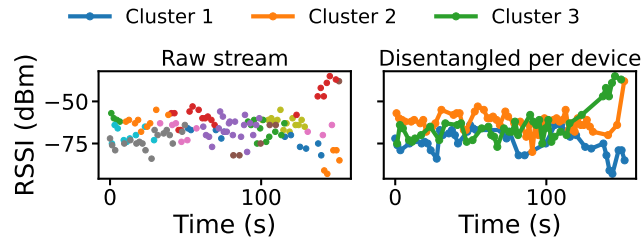
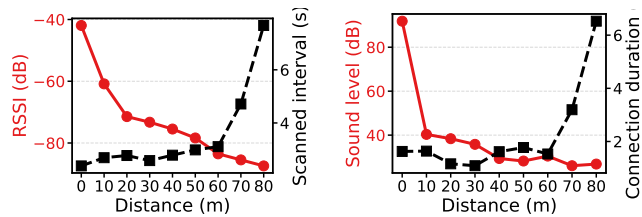


Figure 7: E5, clustering (three devices): from the fragmented rotated-MAC stream (left) the temporal-overlap + RSSI method recovers the three per-device trajectories (right).

(E6) Maximum attack distance (paper Figures 14–15) [10 person-min]. Two field sweeps over the attacker-to-device distance (0–80 m): the BLE discovery channel and the non-owner sound trigger. **Preparation:** `cd reproduce/6_max_distance`.

Execution: `python plot_max_distance.py`. **Results:** two dual-axis plots (Figure 8). (a) RSSI decays smoothly while the scanned advertising interval stays short then rises sharply beyond ~ 60 m (discoverable out to tens of metres); (b) the triggered sound is loud near the device (~ 92 dB) and falls to ambient by ~ 30 m as the trigger time grows — reproducing Figures 14–15.



(a) BLE discovery range: RSSI (left) and scanned advertising interval (right) vs distance. **(b) Sound-trigger range:** sound level (left) and connect-and-trigger time (right) vs distance.

Figure 8: E6, maximum attack distance (paper Figures 14–15): the attack's BLE-discovery range (a) and non-owner-sound range (b), swept from 0 to 80 m.

(E7) End-to-end navigation sample traces (paper Figure 17) [5 person-min]. Example walking trajectories drawn from the

end-to-end navigation evaluation (the academic-building runs Figure 17 averages over), one per attack level. **Preparation:** `cd reproduce/7_navigation_traces`.

Execution: `python plot_traces.py`.

Results: three sample trajectories (Figure 9) from the end-to-end evaluation. (a) the Level-1 acoustic walk; (b) the Level-2 RSSI-IMU walk, coloured by the target's RSSI; (c) the Level-3 walk under clustering, where two lost devices rotate their MAC every 30 s (colour = RSSI, marker shape = distinct MAC). Each converges on the target (green Start $\rightarrow$ red End). The Level-2 panel reuses the trace from (E4), as the academic-building Level-2 runs were captured only as screenshots.

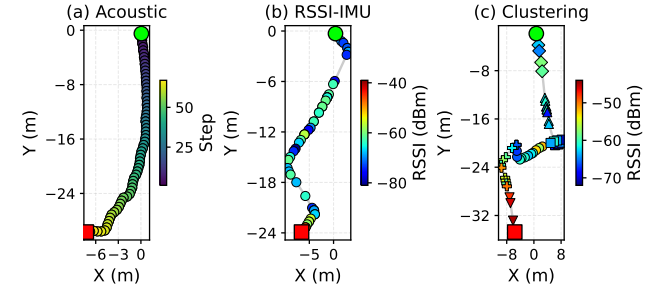


Figure 9: E7, sample end-to-end navigation traces (paper Figure 17), one per attack level, each homing from green Start to red End on the target.